📘 PART 15: C# FUNCTIONS, DELEGATES & EVENTS COMPLETE GUIDE

"Code Ke Blocks, Callbacks, Aur Event System Ka Master Guide"

15.1 🔧 FUNCTIONS / METHODS KYA HAI? (Samjho Pehle)

🔸 Basic Concept:

● TEXT

```
Function = Code ka ek REUSABLE BLOCK jo ek specific kaam karta hai
           Ek baar likho, baar baar use karo!

🧠 Real Life Analogy:
   Function = Recipe (ek baar likhi, baar baar banao)
   Parameters = Ingredients (har baar alag daal sakte ho)
   Return Value = Final Dish (result wapas milta hai)

┌──────────────────────────────────────────────────────┐
│                  FUNCTION ANATOMY                    │
│                                                      │
│   [access] [return_type] [Name]([parameters])        │
│   {                                                  │
│       // Code yahan                                  │
│       return value;  // (agar return type void nahi hai)
│   }                                                  │
│                                                      │
│   Example:                                           │
│   public int AddNumbers(int a, int b)                │
│   {                                                  │
│       return a + b;                                  │
│   }                                                  │
└──────────────────────────────────────────────────────┘

⚠️ Function vs Method:
   Function = Standalone (C/C++ style)
   Method   = Class ke andar ka function (C# mein sab methods hain!)
   But bolchal mein dono same use hote hain
```

15.2 📦 FUNCTION TYPES (Return Types) ⭐⭐⭐

🔸 A. void Functions (Kuch Return Nahi Karte):

● TEXT

```
Kya hai:     Kaam karo but koi value wapas mat do
Use:         Actions, Commands, Setup, UI updates
```

CSHARP

```csharp
public class VoidFunctions : MonoBehaviour
{
    // ===== BASIC VOID FUNCTION =====

    // 1. No parameters, no return
    void SayHello()
    {
        Debug.Log("Hello Bhai!");
    }

    // 2. With parameters, no return
    void TakeDamage(int damage)
    {
        health -= damage;
        Debug.Log($"Ouch! {damage} damage liya! Health: {health}");
    }

    // 3. Multiple parameters
    void MoveTo(float x, float y, float z)
    {
        transform.position = new Vector3(x, y, z);
    }

    // 4. Calling void functions
    void Start()
    {
        SayHello();              // Call karo
        TakeDamage(25);          // Parameter ke saath
        MoveTo(5f, 0f, 10f);     // Multiple parameters
    }

    // ===== REAL GAME EXAMPLES =====

    void Die()
    {
        Debug.Log("Player Dead!");
        GetComponent<Animator>().SetTrigger("Die");
        GetComponent<Collider>().enabled = false;
        GetComponent<Rigidbody>().isKinematic = true;
        GameManager.Instance.GameOver();
    }

    void Respawn(Vector3 spawnPoint)
    {
        transform.position = spawnPoint;
        health = maxHealth;
        gameObject.SetActive(true);
    }

    void PlaySound(AudioClip clip, float volume)
    {
        GetComponent<AudioSource>().PlayOneShot(clip, volume);
    }

    private int health = 100;
    private int maxHealth = 100;
}
```

🔸 B. Return Type Functions (Value Wapas Dete Hain):

TEXT

```
Kya hai:     Kaam karo aur RESULT wapas bhejo
Use:         Calculations, Checks, Data retrieval
⚠️ Rule:     return keyword ke baad function TURANT band ho jaata hai!
```

CSHARP

```csharp
public class ReturnFunctions : MonoBehaviour
{
    // ===== INT RETURN =====

    int GetHealth()
    {
        return health;  // int value wapas bhejo
    }

    int CalculateDamage(int baseDamage, int multiplier)
    {
        return baseDamage * multiplier;
    }

    int GetRandomNumber(int min, int max)
    {
        return Random.Range(min, max);
    }

    // ===== FLOAT RETURN =====

    float GetSpeed()
    {
        return rb.velocity.magnitude;
    }

    float CalculateDistance(Vector3 a, Vector3 b)
    {
        return Vector3.Distance(a, b);
    }

    float GetHealthPercent()
    {
        return (float)currentHealth / maxHealth;
    }

    // ===== BOOL RETURN =====

    bool IsAlive()
    {
        return health > 0;
    }

    bool IsGrounded()
    {
        return Physics.Raycast(transform.position, Vector3.down, 1.1f);
    }
```

```
    bool CanAttack()
    {
        return cooldownTimer ⩽ 0f && mana ⩾ 20f && !isStunned;
    }

    bool IsInRange(Transform target, float range)
    {
        float distance = Vector3.Distance(transform.position, target.position);
        return distance ⩽ range;
    }

    // ===== STRING RETURN =====

    string GetPlayerName()
    {
        return playerName;
    }

    string GetHealthStatus()
    {
        if (health > 75) return "Healthy";
        if (health > 50) return "Wounded";
        if (health > 25) return "Critical";
        return "Dying";
    }

    // ===== VECTOR3 RETURN =====

    Vector3 GetSpawnPosition()
    {
        return spawnPoint.position + Vector3.up * 2f;
    }

    Vector3 GetDirectionToTarget(Transform target)
    {
        return (target.position - transform.position).normalized;
    }

    // ===== GAMEOBJECT RETURN =====

    GameObject FindNearestEnemy()
    {
        GameObject[] enemies = GameObject.FindGameObjectsWithTag("Enemy");
        GameObject nearest = null;
        float minDist = Mathf.Infinity;

        foreach (GameObject enemy in enemies)
        {
            float dist = Vector3.Distance(transform.position, enemy.transform.position);
            if (dist < minDist)
            {
                minDist = dist;
                nearest = enemy;
            }
        }

        return nearest;  // null ho sakta hai agar koi enemy nahi!
    }

    // ===== USING RETURN VALUES =====

    void Update()
```

```
    {
        // Return value ko variable mein store karo
        int dmg = CalculateDamage(10, 3);
        Debug.Log("Damage: " + dmg);  // 30

        // Direct use karo
        if (IsAlive())
        {
            Debug.Log("Zinda hoon!");
        }

        if (CanAttack() && IsInRange(target, 10f))
        {
            Attack();
        }

        // Chain karo
        float dist = CalculateDistance(transform.position, target.position);
        if (dist < 5f)
        {
            Debug.Log("Enemy paas hai!");
        }
    }

    private int health = 100, currentHealth = 100, maxHealth = 100;
    private float cooldownTimer, mana = 50f;
    private bool isStunned;
    private Rigidbody rb;
    private Transform target, spawnPoint;
    private string playerName = "Raju";
    void Attack() { }
}
```

⚠️ Return Ka Rule:

● TEXT

```
⚠️ return ke baad function TURANT EXIT hota hai!
   return ke neeche ka code KABHI nahi chalega!

void CheckHealth()
{
    if (health ⩽ 0)
    {
        Die();
        return;  // ✅ Function yahan RUK jaayega!
        // Neeche ka code NAHI chalega
    }

    // Ye sirf tab chalega jab health > 0 hai
    Debug.Log("Still alive!");
    UpdateHealthBar();
}

int GetScore()
{
    if (isBonusActive)
    {
        return score * 2;  // ✅ Yahan se exit (double score)
    }
```

```
    return score;  // ✅ Normal score (sirf tab jab bonus nahi)
}
```

15.3 📥 PARAMETERS (Input Dena) ⭐⭐⭐

🔸 A. Pass by Value (Default):

TEXT

```
Kya hai:     Parameter ki COPY bheji jaati hai (original change nahi hota)
Use:         Most cases (safe, simple)
```

CSHARP

```
public class PassByValue : MonoBehaviour
{
    void Start()
    {
        int myHealth = 100;
        ApplyDamage(myHealth, 30);

        Debug.Log("Health after: " + myHealth);  // 100! (Change nahi hua!)
        // ↑ Kyunki COPY bheji thi, original safe hai
    }

    void ApplyDamage(int health, int damage)
    {
        health -= damage;  // Ye sirf LOCAL copy ko change karta hai!
        Debug.Log("Inside: " + health);  // 70
    }
}
```

🔸 B. ref Keyword (Reference Se Bhejo):

TEXT

```
Kya hai:     Original variable ka REFERENCE bhejo (changes reflect hote hain!)
Use:         Jab function ke andar value change karni ho aur bahar bhi dikhe
⚠️ Rule:     ref variable ko PEHLE SE initialize karna zaroori hai!
```

CSHARP

```
public class RefDemo : MonoBehaviour
{
    void Start()
    {
        int myHealth = 100;

        ApplyDamageRef(ref myHealth, 30);
        //                 ↑ ref likhna zaroori hai call mein bhi!

        Debug.Log("Health after: " + myHealth);  // 70! (Change hua!)
        // ↑ Kyunki reference bheja tha, original change ho gaya!
```

```
    }

    void ApplyDamageRef(ref int health, int damage)
    {
        health -= damage;  // Ye ORIGINAL variable ko change karta hai!
    }

    // ===== REAL EXAMPLE: Swap Two Values =====

    void Swap(ref int a, ref int b)
    {
        int temp = a;
        a = b;
        b = temp;
    }

    void TestSwap()
    {
        int x = 10, y = 20;
        Swap(ref x, ref y);
        Debug.Log($"x={x}, y={y}");  // x=20, y=10 (swap ho gaya!)
    }

    // ===== REAL EXAMPLE: Modify Stats =====

    void ApplyBuff(ref float speed, ref float damage, float buffMultiplier)
    {
        speed *= buffMultiplier;
        damage *= buffMultiplier;
    }

    void TestBuff()
    {
        float playerSpeed = 5f;
        float playerDamage = 10f;

        ApplyBuff(ref playerSpeed, ref playerDamage, 1.5f);
        Debug.Log($"Speed: {playerSpeed}, Damage: {playerDamage}");
        // Speed: 7.5, Damage: 15
    }
}
```

🔸 C. out Keyword (Output Parameter):

```
TEXT

Kya hai:     Function se VALUE BAHAR nikalo (return jaisa but multiple values!)
Use:         TryGet patterns, Multiple return values, Parsing
⚠️ Difference from ref:
   ref = Variable PEHLE SE initialized hona chahiye
   out = Variable KHAALI ho sakti hai (function ke andar set hogi)
```

```
CSHARP

public class OutDemo : MonoBehaviour
{
    void Start()
    {
        // ===== BASIC OUT =====
```

```
        // 1. Single out parameter
        int result;
        bool success = TryGetScore("Raju", out result);
        //                              ↑ out likhna zaroori!

        if (success)
        {
            Debug.Log("Score: " + result);  // result function ke andar set hua!
        }

        // 2. Inline declaration (C# 7+ - BEST!)
        if (TryGetScore("Shyam", out int shyamScore))
        {
            Debug.Log("Shyam Score: " + shyamScore);
        }

        // 3. Multiple out parameters
        GetPlayerStats(out string name, out int level, out float health);
        Debug.Log($"{name}, Level {level}, HP {health}");

        // ===== REAL EXAMPLES =====

        // 4. Raycast pattern (Unity ka classic out usage!)
        RaycastHit hit;
        if (Physics.Raycast(transform.position, transform.forward, out hit, 100f))
        {
            Debug.Log("Hit: " + hit.collider.name);
            Debug.Log("Distance: " + hit.distance);
            Debug.Log("Point: " + hit.point);
        }

        // 5. TryGetComponent (Modern Unity)
        if (gameObject.TryGetComponent<Rigidbody>(out Rigidbody rb))
        {
            rb.AddForce(Vector3.up * 10f);
        }

        // 6. Dictionary TryGetValue
        Dictionary<string, int> inventory = new Dictionary<string, int>();
        inventory.Add("Sword", 1);

        if (inventory.TryGetValue("Sword", out int count))
        {
            Debug.Log("Sword count: " + count);
        }

        // 7. int.TryParse (Safe parsing)
        string input = "42";
        if (int.TryParse(input, out int parsedNumber))
        {
            Debug.Log("Parsed: " + parsedNumber);  // 42
        }
        else
        {
            Debug.Log("Invalid number!");
        }
    }

    // ===== OUT FUNCTION DEFINITIONS =====

    bool TryGetScore(string playerName, out int score)
```

```
    {
        if (playerName == "Raju")
        {
            score = 100;   // ⚠️ out parameter ko function mein SET karna ZAROORI!
            return true;
        }
        else if (playerName == "Shyam")
        {
            score = 85;
            return true;
        }

        score = 0;  // ⚠️ Har path pe set karna zaroori hai!
        return false;
    }

    void GetPlayerStats(out string name, out int level, out float health)
    {
        name = "Raju";       // ⚠️ Sab set karo!
        level = 5;
        health = 85.5f;
    }
}
```

⚠️ ref vs out Comparison:

● TEXT

```
┌──────────────┬──────────────────────┬──────────────────────┐
│              │         ref          │         out          │
├──────────────┼──────────────────────┼──────────────────────┤
│ Initialize   │ ✅ Pehle se karo     │ ❌ Zaroori nahi      │
│ Set in func  │ ⚠️ Optional          │ ✅ ZAROORI           │
│ Direction    │ IN + OUT (dono)      │ OUT only (bahar)     │
│ Use          │ Modify existing      │ Return extra values  │
│ Example      │ Swap, Buff           │ TryGet, Raycast      │
└──────────────┴──────────────────────┴──────────────────────┘

🧠 Yaad rako:
   ref = "Ye lo, isko change karo, wapas do" (Two-way)
   out = "Mujhe kuch nahi dena, bas result bhejo" (One-way out)
```

🔸 D. params Keyword (Variable Arguments):

● TEXT

```
Kya hai:     Kitne bhi arguments bhej sakte ho (array auto-ban jaata hai)
Use:         Logging, Damage multiple targets, Flexible functions
⚠️ Rule:     params hamesha LAST parameter hona chahiye!
```

● CSHARP

```
public class ParamsDemo : MonoBehaviour
{
    void Start()
    {
        // ===== BASIC PARAMS =====
```

```
        // 1. Kitne bhi numbers bhejo
        int sum1 = AddAll(1, 2, 3);              // 6
        int sum2 = AddAll(10, 20, 30, 40, 50);   // 150
        int sum3 = AddAll(5);                     // 5
        int sum4 = AddAll();                      // 0 (empty!)

        // 2. Strings
        PrintAll("Hello", "World", "Unity", "C#");

        // 3. GameObjects
        DamageAll(25, enemy1, enemy2, enemy3);
    }

    // ===== PARAMS FUNCTIONS =====

    int AddAll(params int[] numbers)
    {
        int total = 0;
        foreach (int num in numbers)
        {
            total += num;
        }
        return total;
    }

    void PrintAll(params string[] messages)
    {
        foreach (string msg in messages)
        {
            Debug.Log(msg);
        }
    }

    void DamageAll(int damage, params GameObject[] targets)
    {
        foreach (GameObject target in targets)
        {
            if (target != null && target.TryGetComponent<EnemyHealth>(out var health))
            {
                health.TakeDamage(damage);
            }
        }
    }

    // ===== REAL EXAMPLE: Log System =====

    void Log(string category, params object[] data)
    {
        string message = $"[{category}] ";
        foreach (var d in data)
        {
            message += d.ToString() + " | ";
        }
        Debug.Log(message);
    }

    void TestLog()
    {
        Log("Combat", "Player", "Hit", "Enemy", 25, "Damage");
        // Output: [Combat] Player | Hit | Enemy | 25 | Damage |
    }
```

```
    private GameObject enemy1, enemy2, enemy3;
}
```

🔸 E. Default Parameter Values:

TEXT

```
Kya hai:     Parameter ka DEFAULT value set karo (na bhejo toh default use hoga)
Use:         Optional settings, Flexible functions
```

CSHARP

```
public class DefaultParams : MonoBehaviour
{
    void Start()
    {
        // ===== DEFAULT PARAMETERS =====

        // 1. Basic defaults
        SpawnEnemy("Goblin");                    // level=1, health=100 (defaults)
        SpawnEnemy("Orc", 5);                    // level=5, health=100
        SpawnEnemy("Dragon", 10, 500);           // level=10, health=500

        // 2. All optional
        PlaySound();                             // volume=1, pitch=1
        PlaySound(shootClip, 0.5f);              // volume=0.5, pitch=1
        PlaySound(shootClip, 0.8f, 1.2f);        // volume=0.8, pitch=1.2

        // 3. Named arguments (order matter nahi karta!)
        SpawnEnemy(health: 200, name: "Boss", level: 15);
        // ↑ Order ulta hai but kaam karega! Named arguments se
    }

    // ===== FUNCTIONS WITH DEFAULTS =====

    void SpawnEnemy(string name, int level = 1, int health = 100)
    {
        Debug.Log($"Spawned {name} (Lv.{level}, HP:{health})");
    }

    void PlaySound(AudioClip clip = null, float volume = 1f, float pitch = 1f)
    {
        if (clip == null) return;
        AudioSource audio = GetComponent<AudioSource>();
        audio.pitch = pitch;
        audio.PlayOneShot(clip, volume);
    }

    // ===== REAL EXAMPLES =====

    void TakeDamage(int amount, bool isCritical = false, string source = "Unknown")
    {
        int finalDamage = isCritical ? amount * 2 : amount;
        health -= finalDamage;
        Debug.Log($"-{finalDamage} HP from {source}" + (isCritical ? " CRITICAL!" : ""));
    }

    void ShowMessage(string text, float duration = 2f, Color color = default)
```

```
    {
        if (color == default) color = Color.white;
        // Show UI message
    }

    void SpawnEffect(Vector3 position, Quaternion rotation = default, float scale = 1f)
    {
        if (rotation == default) rotation = Quaternion.identity;
        // Spawn VFX
    }

    private int health = 100;
    private AudioClip shootClip;
}
```

15.4 🔄 FUNCTION OVERLOADING (Same Name, Different Parameters) ⭐⭐

🔸 Kya Hai:

TEXT

```
Kya hai:     EK HI NAAM ke multiple functions, but ALAG parameters
Use:         Same kaam different tarike se karna
🧠 Real Life: "Chai banao" - Adrak wali, Elaichi wali, Masala wali
              (Sab "Chai" hai but alag ingredients)
```

CSHARP

```
public class OverloadDemo : MonoBehaviour
{
    // ===== OVERLOADED FUNCTIONS =====

    // 1. Different parameter COUNT
    void Attack()
    {
        Debug.Log("Basic attack! Damage: 10");
    }

    void Attack(int damage)
    {
        Debug.Log($"Attack! Damage: {damage}");
    }

    void Attack(int damage, bool isCritical)
    {
        int finalDmg = isCritical ? damage * 2 : damage;
        Debug.Log($"Attack! Damage: {finalDmg}" + (isCritical ? " CRIT!" : ""));
    }

    // 2. Different parameter TYPES
    void TakeDamage(int amount)
    {
        health -= amount;
        Debug.Log($"Took {amount} physical damage!");
    }
```

```
    void TakeDamage(float amount)
    {
        health -= Mathf.RoundToInt(amount);
        Debug.Log($"Took {amount} magic damage!");
    }

    void TakeDamage(string damageType, int amount)
    {
        switch (damageType)
        {
            case "Fire": health -= amount * 2; break;
            case "Ice": health -= amount; speed *= 0.5f; break;
            case "Poison": StartCoroutine(PoisonTick(amount)); break;
            default: health -= amount; break;
        }
        Debug.Log($"Took {amount} {damageType} damage!");
    }

    // 3. Real Example: Spawn Functions
    GameObject Spawn(GameObject prefab)
    {
        return Instantiate(prefab);
    }

    GameObject Spawn(GameObject prefab, Vector3 position)
    {
        return Instantiate(prefab, position, Quaternion.identity);
    }

    GameObject Spawn(GameObject prefab, Vector3 position, Quaternion rotation)
    {
        return Instantiate(prefab, position, rotation);
    }

    GameObject Spawn(GameObject prefab, Transform parent)
    {
        return Instantiate(prefab, parent);
    }

    // Usage:
    void Start()
    {
        Attack();                   // Basic (10 damage)
        Attack(25);                 // 25 damage
        Attack(25, true);           // 50 damage (critical!)

        TakeDamage(10);             // Physical
        TakeDamage(15.5f);          // Magic
        TakeDamage("Fire", 20);     // Fire (40 damage!)

        Spawn(enemyPrefab);
        Spawn(enemyPrefab, spawnPos);
        Spawn(enemyPrefab, spawnPos, spawnRot);
        Spawn(enemyPrefab, parentTransform);
    }

    private int health = 100;
    private float speed = 5f;
    private GameObject enemyPrefab;
    private Vector3 spawnPos;
    private Quaternion spawnRot;
```

```
    private Transform parentTransform;
    IEnumerator PoisonTick(int dmg) { yield break; }
}
```

15.5 ⚡ LAMBDA EXPRESSIONS (Anonymous Functions) ⭐⭐⭐

🔸 Kya Hai:

TEXT

```
Kya hai:     Bina naam ka CHHOTA function (one-liner!)
Syntax:      (parameters) => expression
Use:         Callbacks, LINQ, Events, Short functions
🧠 Real Life: Sticky note pe likha chhota instruction
```

CSHARP

```
public class LambdaDemo : MonoBehaviour
{
    void Start()
    {
        // ===== BASIC LAMBDA =====

        // 1. No parameters
        System.Action greet = () => Debug.Log("Hello!");
        greet();  // "Hello!"

        // 2. One parameter
        System.Action<string> sayName = (name) => Debug.Log("Name: " + name);
        sayName("Raju");  // "Name: Raju"

        // 3. One parameter (shortcut - brackets hatao)
        System.Action<int> printNum = n => Debug.Log("Num: " + n);
        printNum(42);  // "Num: 42"

        // 4. Two parameters
        System.Func<int, int, int> add = (a, b) => a + b;
        int result = add(10, 20);  // 30

        // 5. Multi-line lambda
        System.Action<int, int> complex = (a, b) =>
        {
            int sum = a + b;
            int product = a * b;
            Debug.Log($"Sum: {sum}, Product: {product}");
        };
        complex(5, 3);  // Sum: 8, Product: 15

        // ===== REAL GAME EXAMPLES =====

        // 6. Button click (inline)
        // playButton.onClick.AddListener(() => SceneManager.LoadScene("Game"));

        // 7. Coroutine with lambda condition
        // yield return new WaitUntil(() => health <= 0);

        // 8. Sort with lambda
```

```
        List<int> scores = new List<int> {90, 30, 70, 50, 80};
        scores.Sort((a, b) => b.CompareTo(a));  // Descending order
        // scores: 90, 80, 70, 50, 30

        // 9. Find with lambda
        int firstAbove75 = scores.Find(s => s > 75);  // 90

        // 10. RemoveAll with lambda
        scores.RemoveAll(s => s < 60);  // 30, 50 removed

        // 11. LINQ with lambda
        using System.Linq;
        var highScores = scores.Where(s => s >= 70).OrderByDescending(s => s);
    }
}
```

15.6 📡 DELEGATES (Function Pointers) ⭐⭐⭐

🔸 A. Kya Hai Delegate:

● TEXT

```
Kya hai:     FUNCTION ka REFERENCE store karne wala variable
             Matlab: Function ko variable mein rako, baad mein call karo!
Use:         Callbacks, Plugin systems, Flexible code architecture
🧠 Real Life:
   Normal call = "Raju ko bolo kaam karo" (direct)
   Delegate    = "Ye phone number hai, jab zaroorat ho call karo" (indirect)

┌──────────────────────────────────────────────────────┐
│                   DELEGATE FLOW                      │
│                                                      │
│  1. Define:  delegate void MyAction(int damage);     │
│  2. Assign:  MyAction action = TakeDamage;           │
│  3. Call:    action(25);  // TakeDamage(25) chalega! │
│                                                      │
│  Function ko variable ki tarah use karo!             │
└──────────────────────────────────────────────────────┘
```

🔸 B. Delegate Declaration & Usage:

● CSHARP

```
public class DelegateDemo : MonoBehaviour
{
    // ===== STEP 1: DELEGATE DEFINE KARO =====
    // (Class ke bahar ya andar)

    public delegate void DamageAction(int damage);
    public delegate float CalculateFunc(float a, float b);
    public delegate bool ConditionCheck();

    // ===== STEP 2: FUNCTIONS BANAo (Signature match honi chahiye!) =====

    void TakePhysicalDamage(int damage)
    {
```

```
        health -= damage;
        Debug.Log($"Physical: -{damage} HP");
    }

    void TakeMagicDamage(int damage)
    {
        health -= damage * 2;
        Debug.Log($"Magic: -{damage * 2} HP");
    }

    void ShowDamageNumber(int damage)
    {
        Debug.Log($"💥 {damage} damage!");
    }

    float Add(float a, float b) { return a + b; }
    float Multiply(float a, float b) { return a * b; }

    bool IsAlive() { return health > 0; }
    bool IsStrong() { return health > 75; }

    // ===== STEP 3: USE KARO =====

    void Start()
    {
        // Single function assign
        DamageAction myAction = TakePhysicalDamage;
        myAction(25);  // "Physical: -25 HP"

        // Function change karo!
        myAction = TakeMagicDamage;
        myAction(25);  // "Magic: -50 HP"

        // ===== MULTICAST DELEGATE (Multiple functions!) =====

        DamageAction combo = TakePhysicalDamage;
        combo += ShowDamageNumber;  // Doosra function JOD do!
        combo += PlayHitSound;      // Teesra bhi!

        combo(30);
        // ↑ TEENO functions ek saath chalte hain!
        // "Physical: -30 HP"
        // "💥 30 damage!"
        // "Hit sound played!"

        // Function HATAO
        combo -= ShowDamageNumber;
        combo(20);
        // ↑ Sirf TakePhysicalDamage + PlayHitSound chalega

        // ===== CALCULATOR EXAMPLE =====

        CalculateFunc calc = Add;
        Debug.Log(calc(10, 5));  // 15

        calc = Multiply;
        Debug.Log(calc(10, 5));  // 50

        // ===== CONDITION CHECK =====

        ConditionCheck check = IsAlive;
        if (check())
```

```
        {
            Debug.Log("Player alive!");
        }
    }

    void PlayHitSound(int damage)
    {
        Debug.Log("Hit sound played!");
    }

    private int health = 100;
}
```

15.7 📢 EVENTS (Safe Delegates) ⭐⭐⭐

🔸 A. Kya Hai Event:

TEXT

```
Kya hai:     Delegate ka SAFER version
             Bahar se sirf SUBSCRIBE (+=) aur UNSUBSCRIBE (-=) kar sakte ho
             Directly invoke NAHI kar sakte (sirf owner class kar sakti hai)
Use:         Game events, UI callbacks, Decoupled architecture
🧠 Real Life:
   Delegate = Open microphone (koi bhi bol sakta hai)
   Event    = Radio station (sirf station broadcast karta hai, tum sirf sun sakte ho)

┌──────────────────────────────────────────────────────┐
│                   EVENT PATTERN                      │
│                                                      │
│  PUBLISHER (Jo event fire karta hai):                │
│    public event Action onPlayerDeath;               │
│    onPlayerDeath?.Invoke();                          │
│                                                      │
│  SUBSCRIBER (Jo event sunta hai):                    │
│    Player.onPlayerDeath += HandleDeath;              │
│    Player.onPlayerDeath -= HandleDeath;              │
│                                                      │
│  🧠 "Mujhe batao jab kuch ho, main react karunga"    │
└──────────────────────────────────────────────────────┘
```

🔸 B. Event Declaration & Usage:

CSHARP

```
// ===== PUBLISHER CLASS (Event Fire Karti Hai) =====

public class PlayerHealth : MonoBehaviour
{
    public int currentHealth = 100;
    public int maxHealth = 100;

    // ===== EVENT DECLARATIONS =====

    // Method 1: System.Action (BEST for most cases!)
    public event Action onDeath;                    // No parameters
```

```
    public event Action<int> onHealthChanged;       // 1 parameter
    public event Action<int, int> onDamageTaken;    // 2 parameters
    public event Action<float> onHealthPercentChanged;

    // Method 2: Custom delegate
    public delegate void DeathEvent(PlayerHealth player);
    public event DeathEvent onPlayerDeath;

    // Method 3: System.EventHandler (Standard C# pattern)
    public event System.EventHandler onRespawn;

    // ===== FIRE EVENTS =====

    public void TakeDamage(int damage)
    {
        int oldHealth = currentHealth;
        currentHealth -= damage;
        currentHealth = Mathf.Max(currentHealth, 0);

        // Fire health changed event
        onHealthChanged?.Invoke(currentHealth);
        // ↑ ?. = Null check (agar koi subscriber nahi toh crash nahi hoga!)

        onDamageTaken?.Invoke(damage, currentHealth);

        float percent = (float)currentHealth / maxHealth;
        onHealthPercentChanged?.Invoke(percent);

        if (currentHealth <= 0 && oldHealth > 0)
        {
            Die();
        }
    }

    void Die()
    {
        Debug.Log("Player Dead!");
        onDeath?.Invoke();
        onPlayerDeath?.Invoke(this);
    }

    public void Respawn()
    {
        currentHealth = maxHealth;
        onRespawn?.Invoke(this, System.EventArgs.Empty);
        onHealthChanged?.Invoke(currentHealth);
    }
}
```

● CSHARP

```
// ===== SUBSCRIBER CLASS (Event Sunti Hai) =====

public class UIManager : MonoBehaviour
{
    public PlayerHealth playerHealth;
    public Image healthBar;
    public GameObject deathScreen;

    void OnEnable()
```

```
    {
        // ===== SUBSCRIBE (+=) =====
        // Jab ye object active ho, events sunna shuru karo

        playerHealth.onDeath += HandlePlayerDeath;
        playerHealth.onHealthChanged += UpdateHealthBar;
        playerHealth.onDamageTaken += ShowDamageEffect;
        playerHealth.onHealthPercentChanged += UpdateHealthColor;
    }

    void OnDisable()
    {
        // ===== UNSUBSCRIBE (-=) =====
        // ⚠️ YE ZAROORI HAI! Warna MEMORY LEAK!

        playerHealth.onDeath -= HandlePlayerDeath;
        playerHealth.onHealthChanged -= UpdateHealthBar;
        playerHealth.onDamageTaken -= ShowDamageEffect;
        playerHealth.onHealthPercentChanged -= UpdateHealthColor;
    }

    // ===== EVENT HANDLERS =====

    void HandlePlayerDeath()
    {
        Debug.Log("UI: Player mar gaya! Game Over screen dikhao!");
        deathScreen.SetActive(true);
        Time.timeScale = 0f;
    }

    void UpdateHealthBar(int newHealth)
    {
        healthBar.fillAmount = (float)newHealth / playerHealth.maxHealth;
        Debug.Log($"UI: Health updated to {newHealth}");
    }

    void ShowDamageEffect(int damage, int remainingHealth)
    {
        Debug.Log($"UI: -{damage} damage! Remaining: {remainingHealth}");
        // Screen flash red, damage number popup, etc.
        StartCoroutine(FlashRed());
    }

    void UpdateHealthColor(float percent)
    {
        if (percent > 0.6f) healthBar.color = Color.green;
        else if (percent > 0.3f) healthBar.color = Color.yellow;
        else healthBar.color = Color.red;
    }

    IEnumerator FlashRed()
    {
        yield return new WaitForSeconds(0.1f);
    }
}
```

● CSHARP

```
// ===== ANOTHER SUBSCRIBER (Audio Manager) =====
```

```
public class AudioReactor : MonoBehaviour
{
    public PlayerHealth playerHealth;
    public AudioClip deathSound;
    public AudioClip hitSound;

    void OnEnable()
    {
        playerHealth.onDeath += PlayDeathSound;
        playerHealth.onDamageTaken += PlayHitSound;
    }

    void OnDisable()
    {
        playerHealth.onDeath -= PlayDeathSound;
        playerHealth.onDamageTaken -= PlayHitSound;
    }

    void PlayDeathSound()
    {
        AudioSource.PlayClipAtPoint(deathSound, transform.position);
    }

    void PlayHitSound(int damage, int remaining)
    {
        GetComponent<AudioSource>().PlayOneShot(hitSound);
    }
}
```

🔸 C. Static Events (Global Events):

● CSHARP

```
// ===== GLOBAL EVENT MANAGER =====

public class GameEvents
{
    // Static events = Kisi bhi script se access karo!
    public static event Action onGameStart;
    public static event Action onGamePause;
    public static event Action onGameResume;
    public static event Action onGameOver;
    public static event Action<int> onScoreChanged;
    public static event Action<int> onLevelComplete;
    public static event Action<string> onAchievementUnlocked;

    // Static fire methods
    public static void FireGameStart() => onGameStart?.Invoke();
    public static void FireGamePause() => onGamePause?.Invoke();
    public static void FireGameOver() => onGameOver?.Invoke();
    public static void FireScoreChanged(int score) => onScoreChanged?.Invoke(score);
    public static void FireLevelComplete(int level) => onLevelComplete?.Invoke(level);
}

// ===== USAGE ANYWHERE =====

public class ScoreManager : MonoBehaviour
{
    private int score = 0;
```

```
    void Start()
    {
        GameEvents.onGameStart += ResetScore;
        GameEvents.onGameOver += SaveHighScore;
    }

    void OnDestroy()
    {
        GameEvents.onGameStart -= ResetScore;
        GameEvents.onGameOver -= SaveHighScore;
    }

    public void AddScore(int points)
    {
        score += points;
        GameEvents.FireScoreChanged(score);  // Global event fire!
    }

    void ResetScore() { score = 0; }
    void SaveHighScore() { PlayerPrefs.SetInt("HighScore", score); }
}

// Another script listening to same event:
public class AchievementSystem : MonoBehaviour
{
    void OnEnable()
    {
        GameEvents.onScoreChanged += CheckAchievements;
    }

    void OnDisable()
    {
        GameEvents.onScoreChanged -= CheckAchievements;
    }

    void CheckAchievements(int score)
    {
        if (score ⩾ 1000)
        {
            GameEvents.onAchievementUnlocked?.Invoke("Score Master!");
        }
    }
}
```

15.8 🎯 ACTION & FUNC (Built-in Delegates) ⭐⭐⭐

🔸 A. System.Action (Void Delegates):

```
TEXT

Kya hai:     Unity/C# ka built-in delegate (khud ka delegate nahi banana padta!)
Variants:    Action (0 params), Action<T> (1 param), Action<T1,T2> (2 params)...
Max Params:  16 tak
```

CSHARP

```csharp
using System;

public class ActionDemo : MonoBehaviour
{
    // ===== ACTION DECLARATIONS =====

    Action onJump;                              // No parameters
    Action<int> onDamage;                       // 1 int parameter
    Action<int, string> onItemPickup;           // 2 parameters
    Action<Vector3, float, bool> onExplosion;   // 3 parameters

    void Start()
    {
        // ===== ASSIGN FUNCTIONS =====

        onJump = HandleJump;
        onDamage = HandleDamage;
        onItemPickup = HandleItemPickup;

        // Lambda se bhi assign kar sakte ho:
        onExplosion = (pos, force, isFire) =>
        {
            Debug.Log($"Explosion at {pos}, Force: {force}, Fire: {isFire}");
        };

        // ===== INVOKE =====

        onJump?.Invoke();
        onDamage?.Invoke(25);
        onItemPickup?.Invoke(1, "Sword");
        onExplosion?.Invoke(Vector3.zero, 100f, true);

        // ===== MULTICAST (+=) =====

        onDamage += ShowDamageUI;
        onDamage += PlayHitSound;
        onDamage += CheckDeath;

        onDamage?.Invoke(50);
        // ↑ Teeno functions chalte hain!
    }

    void HandleJump() { Debug.Log("Jump!"); }
    void HandleDamage(int dmg) { Debug.Log($"Damage: {dmg}"); }
    void HandleItemPickup(int id, string name) { Debug.Log($"Picked: {name}"); }
    void ShowDamageUI(int dmg) { }
    void PlayHitSound(int dmg) { }
    void CheckDeath(int dmg) { }
}
```

🔸 B. System.Func (Return Value Delegates):

TEXT

```
Kya hai:     Action jaisa but RETURN VALUE hota hai
Syntax:      Func<ReturnType> ya Func<ParamType, ReturnType>
⚠️ Note:     LAST type parameter hamesha RETURN TYPE hota hai!
```

CSHARP

```csharp
using System;

public class FuncDemo : MonoBehaviour
{
    void Start()
    {
        // ===== FUNC DECLARATIONS =====

        // Func<ReturnType> (no parameters)
        Func<int> getRandomScore = () => Random.Range(0, 100);
        int score = getRandomScore();

        // Func<ParamType, ReturnType> (1 parameter)
        Func<int, bool> isEven = (n) => n % 2 == 0;
        Debug.Log(isEven(4));   // true
        Debug.Log(isEven(7));   // false

        // Func<Param1, Param2, ReturnType> (2 parameters)
        Func<int, int, int> add = (a, b) => a + b;
        Debug.Log(add(10, 20));  // 30

        Func<float, float, float> distance = (a, b) => Mathf.Abs(a - b);
        Debug.Log(distance(10f, 3f));  // 7

        // Func<string, int> (string in, int out)
        Func<string, int> getLength = (s) => s.Length;
        Debug.Log(getLength("Hello"));  // 5

        // ===== REAL GAME EXAMPLES =====

        // Damage calculator (different formulas)
        Func<int, int, int> normalDamage = (baseDmg, defense) => baseDmg - defense;
        Func<int, int, int> criticalDamage = (baseDmg, defense) => (baseDmg * 2) - defense;
        Func<int, int, int> magicDamage = (baseDmg, defense) => baseDmg; // Ignores defense

        // Use based on attack type
        Func<int, int, int> currentFormula = normalDamage;
        int finalDmg = currentFormula(50, 10);  // 40

        currentFormula = criticalDamage;
        finalDmg = currentFormula(50, 10);  // 90

        // Condition checker
        Func<bool> canAttack = () => cooldown <= 0 && mana >= 20;
        if (canAttack())
        {
            Debug.Log("Attack!");
        }
```

```
    }

    private float cooldown, mana = 50f;
}
```

⚠️ Action vs Func vs Delegate:

TEXT

```
┌──────────────┬──────────────┬──────────────┬──────────────┐
│              │   delegate   │   Action     │   Func       │
├──────────────┼──────────────┼──────────────┼──────────────┤
│ Return       │ Any          │ void only    │ Any          │
│ Define       │ Manual       │ Built-in     │ Built-in     │
│ Parameters   │ Any          │ 0-16         │ 0-16 (+ ret) │
│ Syntax       │ delegate...  │ Action<T>    │ Func<T,Ret>  │
│ Use          │ Custom types │ Simple void  │ Return value │
│ Example      │ delegate     │ Action<int>  │ Func<int,bool>│
│              │ void MyDel() │              │              │
└──────────────┴──────────────┴──────────────┴──────────────┘

🧠 Rule:
   90% cases mein Action aur Func use karo (built-in, clean!)
   Custom delegate sirf tab banao jab special signature chahiye
```

15.9 🔗 CALLBACKS (Function Pass Karna) ⭐⭐⭐

🔸 Kya Hai:

TEXT

```
Kya hai:     Function ko PARAMETER ke roop mein doosre function mein bhejna
Use:         Async operations, Timers, Animation complete, Level load
🧠 Real Life: "Pizza order karo, jab ready ho toh CALL BACK karo"
```

CSHARP

```
public class CallbackDemo : MonoBehaviour
{
    // ===== BASIC CALLBACK =====

    // Function jo callback accept karta hai
    void DoAfterDelay(float seconds, Action callback)
    {
        StartCoroutine(DelayCoroutine(seconds, callback));
    }

    IEnumerator DelayCoroutine(float seconds, Action callback)
    {
        yield return new WaitForSeconds(seconds);
        callback?.Invoke();  // Callback call karo!
    }

    // Usage:
    void Start()
```

```
    {
        // 3 second baad message dikhao
        DoAfterDelay(3f, () => {
            Debug.Log("3 seconds ho gaye!");
        });

        // 5 second baad enemy spawn karo
        DoAfterDelay(5f, SpawnEnemy);

        // 2 second baad scene load karo
        DoAfterDelay(2f, () => SceneManager.LoadScene("Level2"));
    }

    // ===== CALLBACK WITH PARAMETERS =====

    void LoadData(string url, Action<string> onSuccess, Action<string> onError)
    {
        StartCoroutine(LoadCoroutine(url, onSuccess, onError));
    }

    IEnumerator LoadCoroutine(string url, Action<string> onSuccess, Action<string> onError)
    {
        // Simulated loading
        yield return new WaitForSeconds(2f);

        bool success = Random.value > 0.3f;

        if (success)
        {
            onSuccess?.Invoke("Data loaded successfully!");
        }
        else
        {
            onError?.Invoke("Failed to load data!");
        }
    }

    void TestLoad()
    {
        LoadData(
            "https://api.example.com/data",
            onSuccess: (data) => Debug.Log("✅ " + data),
            onError: (error) => Debug.Log("❌ " + error)
        );
    }

    // ===== ANIMATION CALLBACK =====

    void PlayAnimation(string animName, Action onComplete)
    {
        StartCoroutine(AnimCoroutine(animName, onComplete));
    }

    IEnumerator AnimCoroutine(string animName, Action onComplete)
    {
        GetComponent<Animator>().Play(animName);
        yield return new WaitForSeconds(1.5f);  // Animation duration
        onComplete?.Invoke();
    }

    void TestAnimation()
    {
```

```
        PlayAnimation("Attack", () => {
            Debug.Log("Attack animation complete! Enable movement!");
            canMove = true;
        });
    }

    // ===== LEVEL LOAD CALLBACK =====

    void LoadLevelWithCallback(string levelName, Action onLoaded)
    {
        StartCoroutine(LoadLevelCoroutine(levelName, onLoaded));
    }

    IEnumerator LoadLevelCoroutine(string levelName, Action onLoaded)
    {
        AsyncOperation op = SceneManager.LoadSceneAsync(levelName);
        while (!op.isDone) yield return null;
        onLoaded?.Invoke();
    }

    void SpawnEnemy() { }
    private bool canMove;
}
```

15.10 🎮 COMPLETE REAL GAME EXAMPLES

🔸 Example 1: Complete Event-Driven Architecture

● CSHARP

```
// ===== GAME EVENT SYSTEM =====
public static class GameEvents
{
    public static event Action onGameStart;
    public static event Action onGamePause;
    public static event Action onGameOver;
    public static event Action<int> onScoreChanged;
    public static event Action<int, Vector3> onEnemyKilled;
    public static event Action<string> onLevelLoaded;

    public static void FireGameStart() => onGameStart?.Invoke();
    public static void FireGameOver() => onGameOver?.Invoke();
    public static void FireScoreChanged(int s) => onScoreChanged?.Invoke(s);
    public static void FireEnemyKilled(int pts, Vector3 pos) => onEnemyKilled?.Invoke(pts, pos);
}

// ===== SCORE MANAGER (Listens + Fires) =====
public class ScoreManager : MonoBehaviour
{
    private int score = 0;

    void OnEnable()
    {
        GameEvents.onEnemyKilled += OnEnemyKilled;
        GameEvents.onGameStart += ResetScore;
    }

    void OnDisable()
```

```
    {
        GameEvents.onEnemyKilled -= OnEnemyKilled;
        GameEvents.onGameStart -= ResetScore;
    }

    void OnEnemyKilled(int points, Vector3 pos)
    {
        score += points;
        GameEvents.FireScoreChanged(score);
        SpawnFloatingText($"+{points}", pos);
    }

    void ResetScore() { score = 0; }
    void SpawnFloatingText(string text, Vector3 pos) { }
}

// ===== UI MANAGER (Listens Only) =====
public class GameUI : MonoBehaviour
{
    public TextMeshProUGUI scoreText;
    public GameObject gameOverPanel;

    void OnEnable()
    {
        GameEvents.onScoreChanged += UpdateScoreUI;
        GameEvents.onGameOver += ShowGameOver;
    }

    void OnDisable()
    {
        GameEvents.onScoreChanged -= UpdateScoreUI;
        GameEvents.onGameOver -= ShowGameOver;
    }

    void UpdateScoreUI(int score)
    {
        scoreText.text = $"Score: {score}";
    }

    void ShowGameOver()
    {
        gameOverPanel.SetActive(true);
        Time.timeScale = 0f;
    }
}

// ===== ENEMY (Fires Events) =====
public class EnemyDeath : MonoBehaviour
{
    public int pointValue = 100;

    public void Die()
    {
        GameEvents.FireEnemyKilled(pointValue, transform.position);
        Destroy(gameObject);
    }
}
```

🔸 Example 2: Callback-Based Timer System

● CSHARP

```
public class TimerSystem : MonoBehaviour
{
    public static TimerSystem Instance;
    void Awake() { Instance = this; }

    public void SetTimer(float duration, Action onComplete, Action<float> onTick = null)
    {
        StartCoroutine(TimerCoroutine(duration, onComplete, onTick));
    }

    public void SetRepeatingTimer(float interval, Action onTick, int repeatCount = -1)
    {
        StartCoroutine(RepeatingTimer(interval, onTick, repeatCount));
    }

    IEnumerator TimerCoroutine(float duration, Action onComplete, Action<float> onTick)
    {
        float remaining = duration;
        while (remaining > 0f)
        {
            remaining -= Time.deltaTime;
            onTick?.Invoke(remaining);
            yield return null;
        }
        onComplete?.Invoke();
    }

    IEnumerator RepeatingTimer(float interval, Action onTick, int count)
    {
        int current = 0;
        while (count == -1 || current < count)
        {
            yield return new WaitForSeconds(interval);
            onTick?.Invoke();
            current++;
        }
    }
}

// Usage:
// TimerSystem.Instance.SetTimer(5f,
//     onComplete: () => Debug.Log("Timer done!"),
//     onTick: (remaining) => Debug.Log($"{remaining:F1}s left")
// );
```

🔸 Example 3: Damage System with Delegates

● CSHARP

```
public class DamageSystem : MonoBehaviour
{
    public delegate float DamageModifier(float incomingDamage);

    private List<DamageModifier> modifiers = new List<DamageModifier>();
```

```
    public void AddModifier(DamageModifier mod)
    {
        modifiers.Add(mod);
    }

    public void RemoveModifier(DamageModifier mod)
    {
        modifiers.Remove(mod);
    }

    public float CalculateFinalDamage(float baseDamage)
    {
        float finalDamage = baseDamage;

        foreach (var mod in modifiers)
        {
            finalDamage = mod(finalDamage);
        }

        return Mathf.Max(finalDamage, 0f);
    }
}

// Usage:
// DamageSystem dmg = GetComponent<DamageSystem>();
// dmg.AddModifier((dmg) => dmg * 1.5f);  // +50% damage buff
// dmg.AddModifier((dmg) => dmg - 10f);   // -10 flat armor
// float final = dmg.CalculateFinalDamage(100f);  // (100*1.5) - 10 = 140
```

15.11 ⚠️ COMMON MISTAKES (Galtiyan)

● CSHARP

```
// ❌ GALTI 1: Event unsubscribe bhool jaana
void OnEnable() { player.onDeath += HandleDeath; }
// ❌ OnDisable mein -= nahi kiya = MEMORY LEAK!
// ✅ SAHI:
void OnDisable() { player.onDeath -= HandleDeath; }

// ❌ GALTI 2: Null event invoke
onDeath.Invoke();  // ❌ CRASH agar koi subscriber nahi!
// ✅ SAHI:
onDeath?.Invoke();  // ✅ Null-safe!

// ❌ GALTI 3: ref/out bhool jaana call mein
void Modify(ref int x) { x = 10; }
Modify(myVar);      // ❌ ERROR! ref keyword chahiye!
// ✅ SAHI:
Modify(ref myVar);  // ✅

// ❌ GALTI 4: out parameter set nahi kiya
void GetData(out int result)
{
    if (condition)
    {
        result = 10;
    }
    // ❌ ERROR! else case mein result set nahi hua!
```

```
}
// ✅ SAHI: Har code path pe set karo
void GetData(out int result)
{
    result = 0;  // Default set
    if (condition) result = 10;
}

// ❌ GALTI 5: Lambda mein variable capture issue
for (int i = 0; i < 5; i++)
{
    buttons[i].onClick.AddListener(() => Debug.Log(i));
}
// ❌ Sab buttons "5" print karenge! (i ki final value)
// ✅ SAHI: Local copy banao
for (int i = 0; i < 5; i++)
{
    int index = i;  // ✅ Local copy!
    buttons[i].onClick.AddListener(() => Debug.Log(index));
}

// ❌ GALTI 6: Event += duplicate
void Start()
{
    player.onDeath += HandleDeath;  // Scene reload pe duplicate!
}
// ✅ SAHI: OnEnable/OnDisable use karo
void OnEnable() { player.onDeath += HandleDeath; }
void OnDisable() { player.onDeath -= HandleDeath; }

// ❌ GALTI 7: Void function se return value expect karna
void DoWork() { /* ... */ }
int result = DoWork();  // ❌ ERROR! void return nahi karta!
```

15.12 📋 QUICK REFERENCE TABLE (Part 15)

Functions:

Type	Syntax	Use
Void	void Func()	Actions, Commands
Return	int Func()	Calculations, Checks
Params	void Func(params int[])	Variable arguments
Default	void Func(int x = 5)	Optional parameters
Overload	Same name, diff params	Flexible API

Parameters:

Keyword	Direction	Init Required	Use
(none)	IN (copy)	Yes	Normal pass

Keyword	Direction	Init Required	Use
ref	IN + OUT	✅ Yes	Modify original
out	OUT only	❌ No	Return extra
params	IN (array)	Yes	Variable count

Delegates & Events:

Type	Return	Syntax	Use
delegate	Any	delegate void D()	Custom signature
Action	void	Action<int>	Void callbacks
Func	Any	Func<int,bool>	Return callbacks
event	Any	event Action	Safe pub/sub

Patterns:

Pattern	Code	Use
Subscribe	`event += Handler`	Listen to event
Unsubscribe	`event -= Handler`	Stop listening
Fire	`event?.Invoke()`	Trigger event
Callback	`Action onComplete`	Async completion
Lambda	`() ⇒ code`	Inline function

🧠 PART 15 YAAD KARNE KE TIPS:

```text
1  void = Kuch return nahi, int/float/bool = Value return karo
2  return ke baad function TURANT exit hota hai
3  ref = Original modify (two-way), out = Result nikalo (one-way)
4  params = Kitne bhi arguments bhejo (last parameter!)
5  Default params = Na bhejo toh default use hoga
6  Overloading = Same naam, alag parameters
7  Lambda = () ⇒ code (chhota inline function)
8  Delegate = Function ka variable | Event = Safe delegate
9  Action = void delegate | Func = return delegate
10 Event += subscribe, -= unsubscribe (OnEnable/OnDisable mein!)
```

✅ PART 15 COMPLETE!

SECTION MAP

16.1 🏗️ OOP KYA HAI? (Samjho Pehle) 514
16.2 📦 CLASSES & OBJECTS ⭐⭐⭐ 514
16.3 🔨 CONSTRUCTORS (Object Ka Janam) ⭐⭐⭐ 517
16.4 🔒 ENCAPSULATION & PROPERTIES ⭐⭐⭐
16.5 👨‍👦 INHERITANCE (Parent → Child) ⭐⭐⭐ 519
16.6 🎭 POLYMORPHISM (Ek Naam, Alag Behavior) ⭐⭐⭐ 521 526
16.7 📐 ABSTRACT CLASSES ⭐⭐⭐ 529
16.8 📋 INTERFACES ⭐⭐⭐ 533
16.9 📦 STRUCTS (Lightweight Classes) ⭐⭐ 537
16.10 🏷️ ENUMS (Advanced) ⭐⭐ 539
16.11 🧬 GENERICS (Type-Safe Templates) ⭐⭐⭐
16.12 📁 NAMESPACES ⭐⭐ 542 540
16.13 🎮 COMPLETE REAL GAME EXAMPLES 543
16.14 ⚠️ COMMON MISTAKES (Galtiyan) 545
16.15 📋 QUICK REFERENCE TABLE (Part 16) 546

📘 PART 16: C# OOP COMPLETE GUIDE

"Object-Oriented Programming - Classes, Inheritance, Interfaces, Polymorphism, Generics Ka Master Guide"

16.1 🏗️ OOP KYA HAI? (Samjho Pehle)

🔸 Basic Concept:

TEXT

```
OOP (Object-Oriented Programming) = Code ko REAL WORLD objects ki tarah organize karna

4 Pillars of OOP:
┌──────────────────────────────────────────────────────┐
│                  OOP KE 4 PILLARS                    │
├──────────────┬───────────────────────────────────────┤
│ 1. ENCAPSULATION │ Data ko protect karo (private + public) │
│ 2. INHERITANCE   │ Parent se child properties lo          │
│ 3. POLYMORPHISM  │ Ek naam, alag behavior                 │
│ 4. ABSTRACTION   │ Complex cheezein simple banao          │
└──────────────┴───────────────────────────────────────┘

🧠 Real Life Analogy:
   Class     = Car ka BLUEPRINT (design drawing)
   Object    = Actual CAR (jo road pe chalti hai)
   Encapsulation = Car ka engine hood ke andar (chhupa hua)
   Inheritance   = Sports Car bhi Car hai (parent → child)
   Polymorphism  = "Start" button → Petrol car mein alag, Electric mein alag
   Abstraction   = Steering wheel ghumaao (andar ka complex mechanism nahi jaanna)
```

16.2 📦 CLASSES & OBJECTS ⭐⭐⭐

🔸 A. Class Kya Hai:

TEXT

```
Class = Blueprint / Template (kuch bhi nahi hai abhi, sirf design hai)
Object = Class se bani hui ACTUAL cheez (real instance)

🧠 Real Life:
   Class  = Ghar ka naksha (blueprint)
   Object = Actual ghar (jo ban gaya hai)
   Ek naksha se 100 ghar ban sakte hain!
```

CSHARP

```
// ===== CLASS DEFINITION =====

public class Weapon
{
```